DEEP NEURAL NETWORK COMPRESSION & COMPUTER VISION

Mid-Evaluation Report

Group 9

January 2026

CONTRIBUTERS

1. **Pratik Beniwal(Group Leader)**
2. **Ashwary Rathore**
3. **Ninad Tagade**
4. **Ayan Abbas**
5. **Sarthak Gupta**
6. **Arnav Agarwal**
7. **Pradeep Meena**
8. **Samayraj Meena**
9. **Naveen Ola**
10. **Madhur Kumar**

1 Computer Vision

This report covers the learning undertaken in the first half of our project. It details the progression from fundamental image processing techniques—such as frequency domain analysis and histograms—to advanced feature extraction methods involving texture, shape, and color descriptors essential for modern machine learning pipelines.

2 Image Basics and Frequency Domain Analysis

2.1 Digital Image Representation

A digital image is fundamentally a discrete representation of visual information. Mathematically, an image can be defined as a two-dimensional function $f(x, y)$, where x and y are spatial coordinates, and the amplitude of f at any pair of coordinates (x, y) is called the intensity or gray level of the image at that point.

In a computer, this function is discretized into a matrix of finite elements called pixels.

- **Grayscale Images:** Represented as a 2D matrix where each element stores an intensity value, typically ranging from 0 (black) to 255 (white) for an 8-bit image.
- **Color Images:** Represented as 3D tensors, typically combining three primary color channels: Red, Green, and Blue (RGB). A pixel vector at location (x, y) is denoted as:

$$\mathbf{C}(x, y) = \begin{bmatrix} R(x, y) \\ G(x, y) \\ B(x, y) \end{bmatrix} \tag{1}$$

2.2 The Frequency Domain and Fourier Transforms

While the spatial domain deals with image intensities directly, the frequency domain analyzes the rate of change of these intensities.

2.2.1 The Fourier Transform

The primary tool for converting spatial data to frequency data is the Fourier Transform. For a 2D image of size $M \times N$, the Discrete Fourier Transform (DFT) is given by:

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi(\frac{ux}{M} + \frac{vy}{N})} \tag{2}$$

Here, $F(u, v)$ represents the complex frequency coefficient at coordinates (u, v). The variables u and v correspond to frequency variables.

2.2.2 Interpretation of Frequencies

- **Low Frequencies:** correspond to slowly varying intensity components, such as the overall shape, background, or smooth regions of an object.
- **High Frequencies:** correspond to rapid changes in intensity, such as edges, noise, and fine details.

Visualizing the magnitude spectrum $|F(u,v)|$ usually requires shifting the zero-frequency component (DC component) to the center of the spectrum and applying a logarithmic transform $\log(1+|F(u,v)|)$ to make the details visible.

2.3 Spatial Masks and Logical Operations

Image processing often requires focusing on a Region of Interest (ROI). This is achieved using binary masks. A mask $M(x,y)$ is a binary image where 1 denotes the ROI and 0 denotes the background. The extracted image $g(x,y)$ is obtained via element-wise multiplication:

$$g(x,y) = f(x,y) \times M(x,y) \tag{3}$$

Logical operations such as AND, OR, and XOR are also fundamental for combining different masks to isolate complex shapes or features.

3 Histograms, Color Spaces, and Point Processing

3.1 Image Histograms

An image histogram is a graphical representation of the tonal distribution in a digital image. It plots the number of pixels for each tonal value. For an image with gray levels in the range $[0, L-1]$, the histogram is a discrete function $h(r_k) = n_k$, where r_k is the k-th gray level and n_k is the number of pixels with that gray level.

The normalized histogram, which represents the probability density function (PDF) of intensity levels, is given by:

$$p(r_k) = \frac{n_k}{MN} \tag{4}$$

where MN is the total number of pixels. Histograms are crucial for understanding image properties:

- **Dark Image:** Components concentrated on the low (left) side.
- **Bright Image:** Components concentrated on the high (right) side.
- **Low Contrast:** Narrow histogram concentrated in the middle.
- **High Contrast:** Broad histogram spanning the full range.

3.2 Color Spaces

While RGB is suitable for displays, it is not always ideal for image analysis because color information is correlated with brightness.

3.2.1 HSV Color Space

We utilize the HSV (Hue, Saturation, Value) model, which aligns more closely with human perception:

- **Hue (H):** Represents the dominant color (e.g., Red, Blue). It is an angular dimension, typically $0° - 360°$.

- **Saturation (S):** Represents the purity of the color (white mixing). Low saturation appears washed out or gray.
- **Value (V):** Represents the brightness or intensity of the color.

This separation allows us to perform robust color detection (using Hue) independent of lighting conditions (Value).

3.3 Point Processing Operations

Point processing involves transforming a pixel's value based solely on its original value, independent of neighbors. The general form is $s = T(r)$.

3.3.1 Brightness and Contrast

Brightness and contrast adjustments are linear transformations:

$$g(x, y) = \alpha \cdot f(x, y) + \beta \tag{5}$$

where $\alpha > 0$ controls contrast (gain) and β controls brightness (bias).

3.3.2 Gamma Correction

Gamma correction is a non-linear operation used to decode luminance. The human eye perceives light non-linearly, and gamma correction compensates for this. The formula is:

$$V_{out} = c \cdot V_{in}^{\gamma} \tag{6}$$

- If $\gamma < 1$: Expands dark regions (makes image brighter).
- If $\gamma > 1$: Compresses dark regions (makes image darker).

3.3.3 White Balance

White balance adjusts the intensities of colors (typically RGB scaling) to render neutral colors correctly. A common technique is the "Gray World Assumption," which posits that the average color of a natural scene should be neutral gray.

4 Convolution, Filtering, and Edge Detection

4.1 Convolution and Kernels

Convolution is the fundamental mathematical operation in image processing for filtering. It involves sliding a small matrix called a kernel (or filter/mask) w over the image f. The output pixel is the weighted sum of the neighbors. Mathematically, for a kernel of size $(2a + 1) \times (2b + 1)$:

$$g(x, y) = w * f = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s, t) f(x - s, y - t) \tag{7}$$

The nature of the kernel determines the effect (e.g., blurring, sharpening, edge detection).

4.2 Smoothing and Blurring

Blurring is a low-pass filter operation used to remove noise.

- **Box Filter:** A uniform kernel where all coefficients are equal (e.g., all 1/9 for a 3×3 kernel). It replaces the center pixel with the average of its neighbors.
- **Gaussian Blur:** Uses a kernel that represents a Gaussian hump. This provides a smoother blur that preserves edges better than a box filter. The 2D Gaussian function is:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} \tag{8}$$

4.3 Edge Detection

Edges are significant local changes in image intensity. We detect them by computing the image gradient.

4.3.1 Sobel Operators

The Sobel operator uses two 3×3 kernels to approximate the derivatives in the horizontal (G_x) and vertical (G_y) directions:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \tag{9}$$

The gradient magnitude $|G|$ and direction θ are given by:

$$|G| = \sqrt{G_x^2 + G_y^2}, \quad \theta = \arctan\left(\frac{G_y}{G_x}\right) \tag{10}$$

4.3.2 Canny Edge Detection

The Canny algorithm is a multi-stage process considered the standard for robust edge detection:
Canny Edge detection pipline is -
Grayscale → Smoothing → Edge detection filters → Gradient magnitude & direction → NonMaximum Suppression → Double thresholding → Edge hysteresis → Binary edge map

1. **Noise Reduction:** Apply Gaussian blur to smooth the image.
2. **Gradient Calculation:** Find intensity gradients (G_x, G_y).
3. **Non-Maximum Suppression:** Thin the edges by suppressing non-maximum pixels in the gradient direction.
4. **Hysteresis Thresholding:** Use two thresholds (T_{high}, T_{low}). Strong edges ($> T_{high}$) are accepted. Weak edges (between T_{low} and T_{high}) are accepted only if they are connected to strong edges.

4.4 Sharpening and Unsharp Masking

Sharpening highlights transitions. A common technique is unsharp masking, where a blurred version of the image is subtracted from the original:

$$f_{sharp}(x, y) = f(x, y) + k \cdot (f(x, y) - f_{blur}(x, y)) \tag{11}$$

Here, $(f - f_{blur})$ represents the high-frequency details (mask), and k controls the amount of sharpening.

5 Geometric Feature Detection

5.1 Corner Detection

Corners are points in an image where intensity changes significantly in two orthogonal directions. They are robust features for tracking and matching.

5.1.1 Harris Corner Detector

The Harris detector analyzes the local structure of the image using the second-moment matrix (Structure Tensor) M:

$$M = \sum_{(x,y)\in W} \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} \tag{12}$$

where I_x and I_y are image derivatives. The eigenvalues λ_1, λ_2 of M determine the region type:

- If $\lambda_1 \approx 0$ and $\lambda_2 \approx 0$: Flat region.
- If $\lambda_1 \gg 0$ and $\lambda_2 \approx 0$ (or vice versa): Edge.
- If $\lambda_1 \gg 0$ and $\lambda_2 \gg 0$: Corner.
- R = $\lambda_1\lambda_2$ - k$(\lambda_1 + \lambda_2)^2$

5.2 The Hough Transform

The Hough Transform is a global method for detecting shapes (lines, circles) that can be parameterized mathematically.

5.2.1 Line Detection

Standard line equation $y = mx + c$ fails for vertical lines ($m \to \infty$). Instead, we use the polar representation:

$$\rho = x \cos\theta + y \sin\theta \tag{13}$$

where ρ is the perpendicular distance from the origin to the line, and θ is the angle of the normal.

- **Algorithm:** We create a 2D accumulator array for (ρ, θ). For every edge pixel (x, y), we iterate through all possible θ and calculate ρ. We increment the vote in the accumulator at (ρ, θ). Peaks in the accumulator correspond to lines in the image.

5.2.2 Circle and Ellipse Detection

This extends the accumulator concept to higher dimensions. A circle is defined by $(x - a)^2 + (y - b)^2 = r^2$. This requires a 3D accumulator for parameters (a, b, r). To reduce computational cost, we often assume a fixed radius or use gradient information to limit the search space (Hough Gradient Method).

6 Advanced Feature Extraction (Color, Shape, Texture)

In this session, we focused on extracting meaningful feature vectors from images to distinguish between different objects (e.g., distinguishing an apple from a banana). We categorized these features into Color, Shape, and Texture.

6.1 Color Features

Color is a powerful descriptor for object identity. We extract statistical moments from the HSV color space to characterize objects.

1. **Mean Hue:** Represents the dominant color identity.
2. **Standard Deviation of Hue (σ_h):** Measures color uniformity. A fresh banana has low σ_h (uniform yellow), while a spotted apple has higher σ_h.
3. **Mean Saturation:** Indicates color purity. Fresh fruits generally exhibit high saturation.
4. **Standard Deviation of Saturation (σ_s):** Correlates with surface roughness. Uneven surfaces cause variations in saturation.
5. **Mean Value:** Measures overall brightness, useful for separating dark berries from bright fruits.
6. **Standard Deviation of Value:** Captures the spread of shadows and highlights. Smooth, spherical fruits often have specific specular highlights, affecting this metric.

6.2 Shape Features

Shape features are extracted from the binary mask or the largest contour of the object. They are generally designed to be invariant to rotation and scale.

6.2.1 Geometric Descriptors

- **Area Ratio:** defined as $\frac{\text{Contour Area}}{\text{Image Area}}$. This is a normalized measure of relative size.
- **Aspect Ratio:** defined as $\frac{\text{Bounding Box Width}}{\text{Bounding Box Height}}$. This measures elongation. (e.g., Bananas have high aspect ratio; Apples have ≈ 1).
- **Solidity:** defined as $\frac{\text{Contour Area}}{\text{Convex Hull Area}}$. It measures compactness. Smooth convex fruits have high solidity, while indented or curved fruits have lower solidity.

- **Circularity:** Measures how close the shape is to a perfect circle.

$$\text{Circularity} = \frac{4\pi \cdot \text{Area}}{(\text{Perimeter})^2} \tag{14}$$

  A perfect circle has a value of 1. Values < 1 indicate elongated or irregular shapes.

6.2.2 Hu Moments

Hu Moments are statistical descriptors of the image intensity distribution, analogous to the physical concept of Moment of Inertia. They are invariant to translation, scale, and rotation. While there are 7 Hu moments, we primarily focus on the first two for stability and noise reduction:

- ϕ_1: Describes the overall spread of the shape.
- ϕ_2: Describes the shape variance.

6.3 Texture Features: Local Binary Patterns (LBP)

Texture provides information about the spatial arrangement of color or intensities. LBP is a computationally efficient and illumination-robust texture operator.

6.3.1 The LBP Algorithm

For a center pixel g_c and its neighbors g_p (where $p = 0 \ldots 7$):

1. Compare each neighbor g_p with the center g_c.
2. Assign a binary value 1 if $g_p \geq g_c$, and 0 otherwise.
3. Concatenate these bits to form a binary number (LBP code).

Mathematically:

$$\text{LBP} = \sum_{p=0}^{P-1} s(g_p - g_c) 2^p \tag{15}$$

where the threshold function $s(x)$ is:

$$s(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases} \tag{16}$$

This process generates a "texture image." To use this for classification, we compute the histogram of these LBP codes, which serves as the texture feature vector.

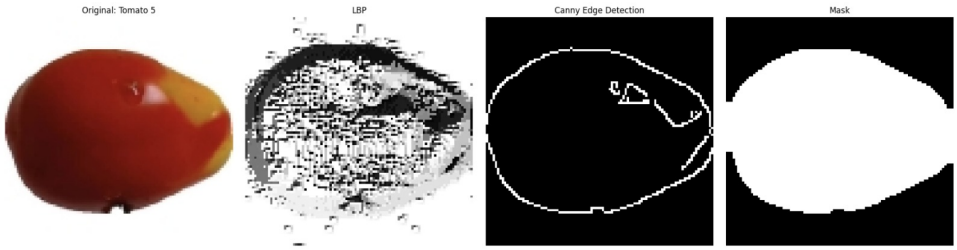

Figure 1: This was the output of the group assignment of the data-loader for the extracting the color features from the particular image

7 Introduction to Machine Learning Models

The main focus of the latter part of the course was on understanding how models learn from data, how training works, why models fail, and how neural networks extend basic learning models from the single perceptrons to the multi-layer perceptrons.

8 Types of Learning

We began by studying the main categories of machine learning.

8.1 Unsupervised Learning

Unsupervised learning works without labels. The goal is to discover patterns or structure in data. Clustering methods such as k-means belong to this category, where similar data points are grouped together.

8.2 Supervised Learning

Supervised learning uses labeled data. Given an input vector x, the model learns to predict an output y. This includes:

8.2.1 Regression and Classification

machine learning problems such as regression and classification can be understood from a probabilistic perspective. instead of directly minimizing an error, we assume an underlying probability distribution for the data and estimate the model parameters that best explain the observed samples.

9 Linear Regression

Linear regression models the relationship between inputs and output using a linear equation:

$$y = w^T x + b \tag{17}$$

where x is the feature vector, w is the weight vector, and b is the bias. The objective of training is to find values of w and b that minimize prediction error.

9.1 Loss Function and Derivation

We use mean squared error (MSE) as the loss function:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{18}$$

Substituting $\hat{y}_i = w^T x_i + b$:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - (w^T x_i + b))^2 \tag{19}$$

This loss function is smooth and differentiable, which allows optimization using gradient-based methods.

9.2 Matrix Form

Let X be the data matrix, y the target vector, and w the weight vector. The MSE can be written as:

$$\text{MSE} = \frac{1}{n}(y - Xw)^T(y - Xw) \tag{20}$$

This compact form simplifies gradient computation and improves computational efficiency.

9.2.1 why we assume the Normal Distribution and how does it lead to the mean squared error??

In linear regression, it is commonly assumed that the target values are generated from a normal (Gaussian) distribution centered around the model prediction:

$$y \sim \mathcal{N}(w^T x, \sigma^2)$$

This assumption implies that the noise in the data is normally distributed with zero mean and variance σ^2. Under this assumption, the shape of the probability distribution is controlled by σ: a larger σ produces a wider and flatter distribution, while a smaller σ results in a sharper peak.

This probabilistic assumption directly leads to the **mean squared error (MSE)** loss used in linear regression.

9.2.2 Maximum Likelihood Estimation

Maximum likelihood estimation (MLE) estimates model parameters by maximizing the probability of observing the given data. For independent samples, the likelihood is:

$$L(\theta) = \prod_i P(y_i \mid x_i, \theta)$$

Taking the logarithm simplifies optimization:

$$\hat{\theta} = \arg\max_{\theta} \log L(\theta)$$

9.2.3 MLE for Linear Regression

Under the Gaussian noise assumption, the conditional distribution is:

$$P(y_i \mid x_i, w) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(y_i - w^T x_i)^2}{2\sigma^2}\right)$$

Maximizing the log-likelihood (ignoring constants) reduces to:

$$\arg\min_w \sum_i (y_i - w^T x_i)^2$$

Thus, MLE for linear regression is equivalent to minimizing MSE.

9.2.4 MLE for Classification

For binary classification, the target variable follows a Bernoulli distribution. Let $p_i = P(y_i = 1)$:

$$P(y_i \mid p_i) = p_i^{y_i}(1 - p_i)^{1-y_i}$$

The log-likelihood over the dataset becomes:

$$\log L = \sum_i [y_i \log p_i + (1 - y_i)\log(1 - p_i)]$$

This yields the binary cross-entropy loss used in logistic regression and neural networks.

9.3 Optimization in Linear Regression

Linear regression parameters are learned by minimizing the mean squared error:

$$\mathcal{L}(w) = \frac{1}{N}\sum_{i=1}^{N}(y_i - w^T x_i)^2$$

The gradient of this loss with respect to the weights is:

$$\nabla_w \mathcal{L} = \frac{2}{N} X^T (Xw - y)$$

This gradient is used in gradient descent to iteratively update the weights and reach the optimal solution.

10 Logistic Regression

Logistic regression is a supervised learning algorithm used for binary classification problems. It models the probability of the positive class using the sigmoid function:

$$P(y = 1 \mid x) = \sigma(w^T x + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

The model is trained by minimizing the binary cross-entropy loss, which allows it to learn a linear decision boundary in the feature space.

11 why we use Gradient descent??

Training a model means minimizing the loss function, which is done using gradient descent.

11.1 Gradient of MSE

Starting from:

$$\text{MSE} = \frac{1}{n}(y - Xw)^T(y - Xw) \tag{21}$$

Taking the derivative with respect to w:

$$\nabla_w \text{MSE} = \frac{2}{n}X^T(Xw - y) \tag{22}$$

This gradient indicates the direction in which the loss increases most rapidly.

11.2 Gradient Descent Update Rule

The weight update rule is:

$$w_{\text{new}} = w_{\text{old}} - \eta \nabla_w \text{MSE} \tag{23}$$

where η is the learning rate. Repeated updates move the model toward minimum loss.

12 K means clustering(UNSUPERVISED LEARNING)

12.1 Problem with Normal K-means Initialization

In standard K-means, cluster centers are initialized randomly. This can result in multiple centroids starting very close to each other, leaving some clusters poorly represented and causing convergence to bad local minima.

12.2 K-means++ Initialization

K-means++ improves centroid initialization by spreading the initial means across the data space instead of choosing them uniformly at random.

12.3 K-means++ Algorithm

1. Choose the first centroid randomly from the dataset.
2. For each data point x_i, compute its distance from the nearest centroid:

$$D(x_i)^2 = \min_j \|x_i - \mu_j\|^2$$

3. Select the next centroid with probability:

$$P(x_i) = \frac{D(x_i)^2}{\sum_k D(x_k)^2}$$

4. Repeat until k centroids are selected.

5. Run standard K-means.

12.4 Why K-means++ Works Better

Points farther from existing centroids have higher selection probability, which ensures well-separated initial centers and leads to faster convergence and improved clustering quality.

12.5 Disadvantages of unsupervised to use instead of supervised

1. Unsupervised learning has no labeled data, so there is no clear ground truth for evaluating performance.

2. The discovered patterns or clusters are often difficult to interpret and may not match real-world classes.

3. Results are highly sensitive to model assumptions and parameter choices such as the number of clusters.

4. The algorithm may learn noise instead of meaningful structure, leading to unreliable results.

5. Overall, unsupervised learning is less effective than supervised learning for prediction tasks.

13 Underfitting, Overfitting, and Regularization

13.1 Learning Objective

In supervised regression, the goal is to learn a function that maps inputs to outputs:

$$y = f(x) = w^T x + b$$

The parameters are learned by minimizing the mean squared error (MSE):

$$\mathcal{L}(w) = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

Minimizing this loss ensures that predictions remain close to the true targets on average. However, low training loss alone does not guarantee good generalization to unseen data.

13.2 Underfitting

Underfitting occurs when the model is too simple to capture the underlying data distribution. For example, fitting a linear model

$$y = mx + c$$

to nonlinear data leads to poor performance.

$$\text{MSE}_{\text{train}} \text{ high}, \quad \text{MSE}_{\text{test}} \text{ high}$$

Underfitting is caused by insufficient model complexity or lack of informative features. It can be reduced by increasing model capacity, such as using polynomial regression:

$$f(x) = w_1 x + w_2 x^2 + \cdots + w_k x^k$$

13.3 Overfitting

Overfitting occurs when a model learns both the data pattern and noise in the training set. Such models perform very well on training data but poorly on test data:

$$\mathrm{MSE}_{\text{train}} \approx 0, \quad \mathrm{MSE}_{\text{test}} \gg 0$$

This typically happens when the model is overly complex and memorizes training samples instead of learning general trends.

13.4 Bias–Variance Tradeoff

The errors in learning models can be explained using bias and variance:

- Underfitting: high bias, low variance
- Overfitting: low bias, high variance

The objective of learning is to find an optimal balance between bias and variance that minimizes total error.

13.5 Regularization

Regularization controls model complexity by penalizing large parameter values. The regularized loss function is given by:

$$\mathcal{L}_{\text{reg}} = \mathrm{MSE} + \lambda \sum_i w_i^2$$

Here, λ is the regularization parameter. Small values of λ allow more complex models, while large values enforce simpler models. Penalizing large weights reduces sensitivity to noise and improves generalization.

13.6 Feature Scaling

Regularization assumes that all features have comparable scales. Therefore, features are standardized using:

$$x' = \frac{x - \mu}{\sigma}$$

where μ is the mean and σ is the standard deviation.

Feature scaling ensures that regularization affects all parameters uniformly.

14 Multilayer Perceptron (MLP)

14.1 Why Do We Use an MLP?

Single-layer models such as linear regression or a single perceptron can only learn linear relationships. These models compute functions of the form:

$$y = w^T x + b$$

which represent straight lines or hyperplanes. However, most real-world data exhibits nonlinear relationships.

The motivation behind using a multilayer perceptron is to overcome this limitation. By stacking multiple layers and introducing nonlinear activation functions, an MLP can learn complex, nonlinear mappings between inputs and outputs.

14.2 What Is a Multilayer Perceptron?

A multilayer perceptron is a feedforward neural network consisting of an input layer, one or more hidden layers, and an output layer. Each neuron performs a linear transformation followed by a nonlinear activation:

$$z = w^T x + b$$

$$h = \phi(z)$$

A commonly used activation function is the Rectified Linear Unit (ReLU), defined as:

$$\mathrm{ReLU}(x) = \max(0, x)$$

Nonlinear activation functions are essential because, without them, multiple stacked layers would still behave like a linear model.

14.3 Advantages of MLP

- **Nonlinear Learning Capability:** MLPs can model complex nonlinear relationships present in data.
- **Universal Approximation:** An MLP with at least one hidden layer and nonlinear activation can approximate any continuous function:

$$f(x) \approx \sum_i w_i \,\mathrm{ReLU}(a_i x + b_i)$$

- **Automatic Feature Learning:** Hidden layers learn intermediate feature representations automatically.
- **Versatility:** MLPs can be applied to regression, classification, and function approximation tasks.

14.4 MLP Procedure

14.4.1 Forward Propagation

During forward propagation, input data is passed through the network layer by layer. For the first hidden layer:

$$h_1 = \text{ReLU}(W_1 x + b_1)$$

For subsequent layers:

$$h_2 = \text{ReLU}(W_2 h_1 + b_2)$$

The final output is computed as:

$$\hat{y} = W_o h + b_o$$

14.4.2 Loss Computation

The error between the predicted output $\hat{y}$ and the true output y is measured using a loss function, commonly the mean squared error:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

14.4.3 Backpropagation

Backpropagation is used to compute gradients of the loss with respect to each parameter. Using the chain rule, gradients are propagated from the output layer back to the input layer:

$$\frac{\partial \mathcal{L}}{\partial w}$$

14.4.4 Parameter Update

Parameters are updated using gradient descent:

$$w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w}$$

where η is the learning rate. This process is repeated over multiple iterations until convergence.

14.5 providing the pseudocode

Algorithm 1 Training a Multilayer Perceptron (MLP)

Require: Training data (X, Y), learning rate η, epochs E
Ensure: Trained weights $\{W_l\}$ and biases $\{b_l\}$

```
Initialize weights W_l and biases b_l randomly
for epoch = 1 to E do
    for each training sample (x, y) ∈ (X, Y) do
                                                    ▷ Forward Propagation
        a_0 ← x
        for l = 1 to L do
            z_l ← W_l a_{l-1} + b_l
            a_l ← ReLU(z_l)
        end for
        ŷ ← a_L
                                                    ▷ Loss Computation
        L ← (y − ŷ)^2
                                                    ▷ Backpropagation
        δ_L ← ∂L/∂a_L
        for l = L to 1 do
            δ_l ← δ_l · ReLU'(z_l)
            ∂L/∂W_l ← δ_l a_{l-1}^T
            ∂L/∂b_l ← δ_l
            δ_{l-1} ← W_l^T δ_l
        end for                                     ▷ Parameter Update
        for l = 1 to L do
            W_l ← W_l − η ∂L/∂W_l
            b_l ← b_l − η ∂L/∂b_l
        end for
    end for
end for
```

14.6 Summary

The multilayer perceptron extends linear models by incorporating multiple layers and nonlinear activations. Through forward propagation, loss computation, backpropagation, and gradient-based optimization, MLPs are able to learn complex patterns and generalize effectively.

15 Convolutional Neural Networks (CNN)

A Convolutional Neural Network (CNN) is a specialized deep learning architecture designed primarily for processing data with a grid-like topology, such as images. Unlike standard Multi-Layer Perceptrons (MLPs), which connect every input neuron to every output neuron resulting in "many weights" and high computational cost, CNNs use convolutional layers to process data more efficiently.

Key defining characteristics include:

Local Connectivity (Filters): Instead of dense matrix multiplication, CNNs use small filters (kernels) that slide across the input to perform mathematical operations (convolution).

Translational Invariance: The network is designed to recognize features regardless of their position in the image.

Dimensionality Reduction: CNNs employ Max Pooling layers to downsample the feature maps, reducing the spatial dimensions and computational load.

Non-Linearity: They typically use ReLU (Rectified Linear Unit) activation functions ($a = \text{ReLU}[Wx + b]$) to introduce non-linearity into the model.

16 Why to use CNN rather than ANN for images ?

The Limitation of Standard Neural Networks (MLPs)

The Computational Problem: If you take an input image (e.g., 100×100 pixels) and flatten it into a vector, you get a massive number of input features1. Connecting these to a hidden layer results in "many weights," making the model "computationally heavy"2.

Approximation: While neural networks act as universal approximators using piecewise linear functions like ReLU ($a = \text{ReLU}[Wx + b]$), the sheer size of the weight matrix for images makes standard MLPs impractical.

The Solution: Convolutional Neural Networks (CNNs)

Translational Invariance: Unlike MLPs, CNNs leverage "translational invariance"4. This means the network can recognize a feature (like an edge or shape) regardless of where it appears in the image.

Filters (Kernels): Instead of connecting every neuron to every pixel, CNNs use small filters (e.g., 2×3 or similar dimensions) that slide across the input data5. This drastically reduces the number of parameters.

17 Convolutional Neural Networks (CNN)

A Convolutional Neural Network (CNN) is a specialized deep learning architecture designed primarily for processing data with a grid-like topology, such as images. Unlike standard Multi-Layer Perceptrons (MLPs), which connect every input neuron to every output neuron resulting in "many weights" and high computational cost, CNNs use convolutional layers to process data more efficiently.

A typical CNN architecture is shown in Figure 2.

Key defining characteristics include:

Local Connectivity (Filters): Instead of dense matrix multiplication, CNNs use small filters (kernels) that slide across the input to perform mathematical operations (convolution).

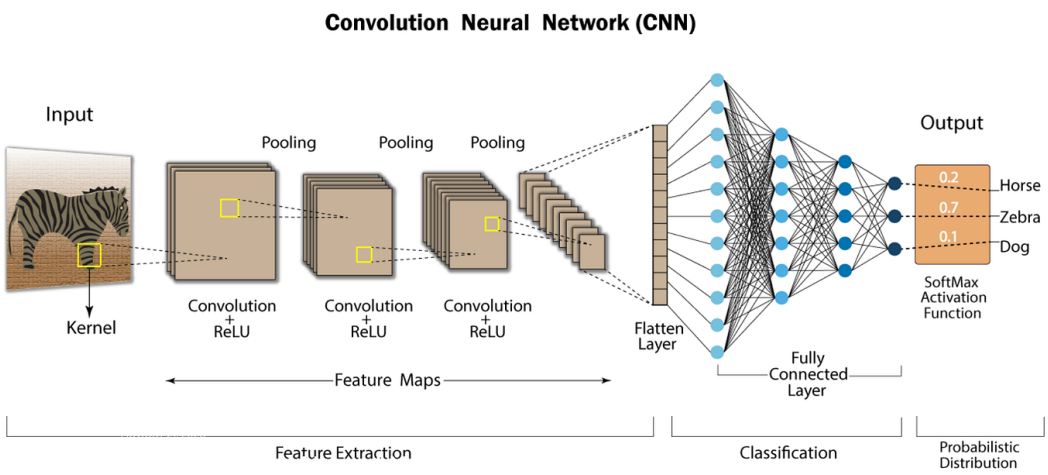

Figure 2: A standard Convolutional Neural Network (CNN) architecture, showing the feature extraction layers (Convolution, ReLU, Pooling) and the final classification layers.

Translational Invariance: The network is designed to recognize features regardless of their position in the image.

Dimensionality Reduction: CNNs employ Max Pooling layers to downsample the feature maps, reducing the spatial dimensions and computational load.

Non-Linearity: They typically use ReLU (Rectified Linear Unit) activation functions ($a = \text{ReLU}[Wx + b]$) to introduce non-linearity into the model.

18 Network Pruning

Network pruning is a compression technique that reduces the redundancy in neural network parameters by removing unimportant connections. This project focuses on **Magnitude-based Pruning**.

18.1 Motivation: Weight Distribution

In a typical trained Convolutional Neural Network (CNN), the distribution of weights often resembles a Gaussian (Normal) distribution centered at zero. As noted in the lecture, a significant portion of weights (often up to 80%) have values very close to zero [102]. These weights contribute minimally to the network's output activations and can be considered redundant.

18.2 The Pruning Algorithm

The process involves selecting a threshold to zero-out these redundant weights. The algorithm proceeds as follows:

1. **Select Threshold (th):** Determine a threshold value based on the desired sparsity level (e.g., the value at the 80th percentile of the weight magnitudes) [104].

2. **Apply Masking:** For every weight w in the layer, apply the following rule:

$$w_{pruned} = \begin{cases} 0 & \text{if } |w| < th \\ w & \text{otherwise} \end{cases} \tag{24}$$

This effectively removes the connection from the network.

18.3 Benefits: Sparsity and Storage

The direct result of this operation is the transformation of a dense weight matrix into a **Sparse Matrix**.

- **Storage Reduction:** Instead of storing every floating-point value in a dense matrix (which consumes significant RAM), we can use sparse formats (like CSR) to store only the non-zero values and their indices. This directly addresses the "RAM size" constraints on edge devices.
- **Computational Efficiency:** Sparse matrix multiplication requires fewer floating-point operations (FLOPs) since multiplications by zero are skipped.

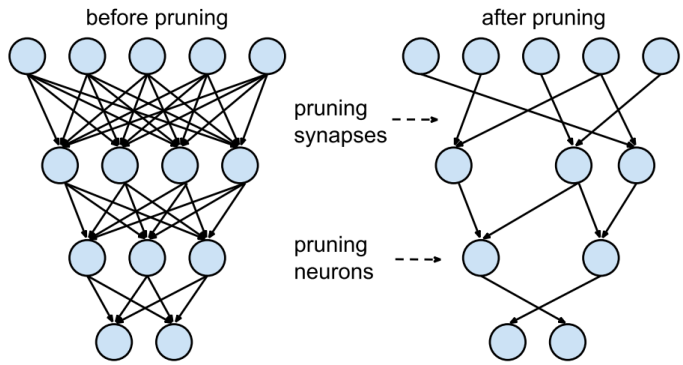

Figure 3: Visual comparison of a neural network before and after pruning. The model on the left is dense with full connectivity. The model on the right after pruning is sparse, with redundant synapses and neurons removed to reduce size and computational complexity.

19 PyTorch Functions and Their Role in Learning

PyTorch is a dynamic deep learning framework that simplifies neural network implementation by combining efficient tensor computation with automatic differentiation. Its design closely follows the mathematical learning process, making model development, experimentation, and debugging intuitive.

19.0.1 Tensors

Tensors are the fundamental data structure in PyTorch, generalizing scalars, vectors, and matrices to higher dimensions. Functions such as `torch.tensor()` and `torch.randn()` are used to create tensors, while `torch.matmul()` performs matrix multiplication. These operations are optimized for parallel computation on GPUs, enabling efficient large-scale training.

19.0.2 Gradient Tracking

PyTorch enables learning by tracking operations on tensors using the `requires_grad=True` flag. When this property is set, PyTorch records all operations involving the tensor, allowing gradients to be computed automatically. Model parameters such as weights and biases are typically created with this setting.

19.0.3 Automatic Differentiation

PyTorch uses the `torch.autograd` engine to compute gradients automatically. During the forward pass, it dynamically constructs a computational graph that stores all operations. This graph is later used to apply the chain rule during backpropagation, removing the need for manual gradient derivations.

19.0.4 Backward Pass

The backward pass is initiated by calling `loss.backward()`. This function computes gradients of the loss with respect to all tensors that have `requires_grad=True`. The computed gradients are stored in the `.grad` attribute of each tensor.

19.0.5 Gradient Control

PyTorch provides the `torch.no_grad()` context manager to disable gradient tracking temporarily. This is commonly used during model evaluation or inference to reduce memory usage and improve computational efficiency.

19.0.6 Parameter Updates

Model parameters are updated using gradients stored in the `.grad` attribute. Optimizers apply gradient descent-based rules of the form:

$$w \leftarrow w - \eta \nabla L$$

where η is the learning rate and ∇L is the gradient of the loss function.

19.0.7 End-to-End Workflow

PyTorch supports the complete training workflow, including data loading using `Dataset` and `DataLoader`, forward pass computation, loss calculation, backpropagation, and parameter optimization. This modular structure provides flexibility and fine-grained control over the learning process.

20 Conclusion

This report presents a comprehensive progression from fundamental image processing concepts to advanced deep learning models used in computer vision. Starting with image representation, frequency analysis, and feature extraction, it builds a strong mathematical and intuitive foundation for understanding visual data. The study then transitions into machine learning models, covering linear and logistic regression, optimization, and

learning theory. Advanced architectures such as MLPs and CNNs are explored to address nonlinearity and image-specific challenges. Finally, model compression through network pruning and efficient implementation using PyTorch highlights the importance of scalability and deployment on resource-constrained systems. Overall, the report integrates theory, mathematics, and practical frameworks into a cohesive learning pipeline for modern computer vision systems